Федеральное автономное образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Факультет Программной инженерии и компьютерной техники
Направление подготовки 09.03.04 Программная инженерия

Дисциплина: «*Базы данных*»

Отчёт
По лабораторной работе №4
Вариант ***2795***

Выполнил:
Крамарь Михаил Романович
P3113

Проверил:
Остапенко Иван Вадимович

г. Санкт-Петербург
2026

Содержание

Текст задания 3
Реализация запросов на SQL 4
Индексы, добавление которых уменьшит время выполнения запроса 5
Планы выполнения запросов 6
Вывод команды EXPLAIN ANALYZE 8
Выводы по работе 9

Текст задания

Лабораторная работа №4

По варианту, выданному преподавателем, составить и выполнить запросы к базе данных "Учебный процесс".

Команда для подключения к базе данных ***ucheb***:

psql -h pg -d ucheb

Составить запросы на языке SQL (пункты 1-2).

Для каждого запроса предложить индексы, добавление которых уменьшит время выполнения запроса (указать таблицы/атрибуты, для которых нужно добавить индексы, написать тип индекса; объяснить, почему добавление индекса будет полезным для данного запроса).

Для запросов 1-2 необходимо составить возможные планы выполнения запросов. Планы составляются на основании предположения, что в таблицах отсутствуют индексы. Из составленных планов необходимо выбрать оптимальный и объяснить свой выбор.
Изменятся ли планы при добавлении индекса и как?

Для запросов 1-2 необходимо добавить в отчет вывод команды EXPLAIN ANALYZE [запрос]

Подробные ответы на все вышеперечисленные вопросы должны присутствовать в отчёте (планы выполнения запросов должны быть нарисованы, ответы на вопросы - представлены в текстовом виде).

1. Сделать запрос для получения атрибутов из указанных таблиц, применив фильтры по указанным условиям:
   Таблицы: Н_ЛЮДИ, Н_СЕССИЯ.
   Вывести атрибуты: Н_ЛЮДИ.ИМЯ, Н_СЕССИЯ.ИД.
   Фильтры (AND):
   a) Н_ЛЮДИ.ФАМИЛИЯ = Иванов.
   b) Н_СЕССИЯ.ИД = 32199.
   c) Н_СЕССИЯ.ИД < 1975.
   Вид соединения: LEFT JOIN.
2. Сделать запрос для получения атрибутов из указанных таблиц, применив фильтры по указанным условиям:
   Таблицы: Н_ЛЮДИ, Н_ВЕДОМОСТИ, Н_СЕССИЯ.
   Вывести атрибуты: Н_ЛЮДИ.ФАМИЛИЯ, Н_ВЕДОМОСТИ.ДАТА, Н_СЕССИЯ.ИД.
   Фильтры (AND):
   a) Н_ЛЮДИ.ФАМИЛИЯ > Иванов.
   b) Н_ВЕДОМОСТИ.ЧЛВК_ИД = 105590.
   Вид соединения: RIGHT JOIN.

Реализация запросов на SQL

Запрос 1:

```
-- Сделать запрос для получения атрибутов из указанных таблиц, применив фильтры по указанным условиям:
-- Таблицы: Н_ЛЮДИ, Н_СЕССИЯ.
-- Вывести атрибуты: Н_ЛЮДИ.ИМЯ, Н_СЕССИЯ.ИД.
-- Фильтры (AND):
-- a) Н_ЛЮДИ.ФАМИЛИЯ = Иванов.
-- b) Н_СЕССИЯ.ИД = 32199.
-- c) Н_СЕССИЯ.ИД < 1975.
-- Вид соединения: LEFT JOIN.

-- запрос сам по себе бредовый, так как очевидно выведет пустую таблицу
SELECT people."ИМЯ" AS "ИМЯ", ses."ИД" AS "ID_СЕССИИ" FROM
"Н_ЛЮДИ" AS people
LEFT OUTER JOIN "Н_СЕССИЯ" AS ses
    ON people."ИД" = ses."ЧЛВК_ИД"
WHERE people."ФАМИЛИЯ" = 'Иванов' AND
    ses."ИД" = 32199 AND
    ses."ИД" < 1975;
```

Запрос 2:

```
-- Сделать запрос для получения атрибутов из указанных таблиц, применив фильтры по указанным условиям:
-- Таблицы: Н_ЛЮДИ, Н_ВЕДОМОСТИ, Н_СЕССИЯ.
-- Вывести атрибуты: Н_ЛЮДИ.ФАМИЛИЯ, Н_ВЕДОМОСТИ.ДАТА, Н_СЕССИЯ.ИД.
-- Фильтры (AND):
-- a) Н_ЛЮДИ.ФАМИЛИЯ > Иванов.
-- b) Н_ВЕДОМОСТИ.ЧЛВК_ИД = 105590.
-- Вид соединения: RIGHT JOIN.

SELECT people."ФАМИЛИЯ", veds."ДАТА", ses."ИД" FROM
"Н_ВЕДОМОСТИ" AS veds
RIGHT OUTER JOIN "Н_ЛЮДИ" AS people
    ON people."ИД" = veds."ЧЛВК_ИД"
RIGHT OUTER JOIN "Н_СЕССИЯ" AS ses
    ON people."ИД" = ses."ЧЛВК_ИД"
WHERE people."ФАМИЛИЯ" > 'Иванов'
    AND veds."ЧЛВК_ИД" = 105590;
```

Индексы, добавление которых уменьшит время выполнения запроса

1 Запрос:

Для первого запроса можно добавить индексы на следующие таблицы и их атрибуты:

- «Н_ЛЮДИ».ФАМИЛИЯ- индекс типа B-Tree
- «Н_СЕССИЯ».ИД - индекс типа B-Tree
- «Н_СЕССИЯ».ЧЛВК_ИД - индекс типа B-Tree

1) Индекс на атрибут «ФАМИЛИЯ» в таблице «Н_ЛЮДИ» ускорит выполнение операции WHERE, так как без индекса придётся просканировать все строки данной таблицы. С индексом, СУБД сможет быстро найти нужные нам строки, значительно ускоряя выполнение нашего запроса.
2) Индекс на атрибут «ИД» в таблице «Н_СЕССИЯ» точно так же позволит ускорить выполнение запроса, так как без него опять же придётся просканировать всю нашу таблицу, чтобы найти нужные нам кортежи. При наличии индекса СУБД быстро найдёт нужные строки в соответствии с запросом.
3) Мы можем оптимизировать выполнение операции объединения таблиц LEFT OUTER JOIN при помощи добавления индекса на атрибут «ЧЛВК_ИД» в таблице «Н_СЕССИЯ». Без индекса будет выполнен вложенный цикл, просматривающий каждую строку таблицы «Н_ЛЮДИ» и для каждой строки ищущий соотвествующий кортеж в таблице «Н_СЕССИЯ». После добавления индекса на атрибут «ЧЛВК_ИД» СУБД сможет использовать поиск по индексу, что позволит мгновенно находить нужные строки в таблице «Н_СЕССИЯ», сокращая общее время выполнения запроса.

2 Запрос:

Для второго запроса можно добавить индексы на следующие таблицы и их атрибуты:

- «Н_ЛЮДИ».ФАМИЛИЯ- индекс типа B-Tree
- «Н_ЛЮДИ».ИД - индекс типа B-Tree
- «Н_ВЕДОМОСТИ».ЧЛВК_ИД - индекс типа B-Tree

1) Индекс на атрибут «ФАМИЛИЯ» в таблице «Н_ЛЮДИ» ускорит выполнение операции WHERE, так как без индекса придётся просканировать все строки данной таблицы. С индексом, СУБД сможет быстро найти нужные нам строки, значительно ускоряя выполнение нашего запроса.
2) Добавляя индекс на атрибут «ЧЛВК_ИД» в таблице «Н_ВЕДОМОСТИ», мы получаем оптимизацию выполнения операции RIGHT OUTER JOIN с таблицей «Н_ЛЮДИ». Без индекса на данном атрибуте, СУБД выполнить вложенный цикл, который будет просматривать каждый кортеж таблицы «Н_ЛЮДИ» и проходить по всей таблице «Н_ВЕДОМОСТИ» в поиске соответствия. С индексом на атрибуте «ЧВЛК_ИД» система сможет применять поиск по индексу, сопоставляя каждой строке из «Н_ЛЮДИ» индекс из «Н_ВЕДОМОСТИ» и по ней находя нужные строки, это значительно сокращает время выполнения нашего запроса, особенно если таблица большая.
3) Аналогично с добавлением индекса на атрибут «ЧЛВК_ИД» в «Н_ВЕДОМОСТИ», добавляем его на атрибут «ИД» в таблице «Н_ЛЮДИ» для оптимизации операции RIGHT JOIN с таблицей «Н_СЕССИЯ». Здесь оправданность и польза добавления индекса точно такая же, как и в пункте 2).

Планы выполнения запросов

Запрос 1:

План 1:
- Полный скан таблицы «Н_ЛЮДИ»
- Полный скан таблицы «Н_СЕССИЯ»
- Соединение таблиц «Н_ЛЮДИ» и «Н_СЕССИЯ» при помощи *Nested Loops Join* по соответствующим атрибутам «ИД» и «ЧЛВК_ИД»
- Конечная фильтрация результата соединения этих таблиц по условию «Н_ЛЮДИ».ФАМИЛИЯ = ‘Иванов’ AND «Н_СЕССИЯ».ИД = 32199 AND «Н_СЕССИЯ».ИД < 1975

План 2:
- Полный скан таблицы «Н_ЛЮДИ» сразу с применением фильтра «Н_ЛЮДИ».ФАМИЛИЯ = ‘Иванов’
- Полный скан таблицы «Н_СЕССИЯ» с применением фильтра «Н_СЕССИЯ».ИД = 32199 AND «Н_СЕССИЯ».ИД < 1975
- Соединение таблиц «Н_ЛЮДИ» и «Н_СЕССИЯ» при помощи *Nested Loops Join* (вложенные циклы)

Оптимальный план:
- Очевидно, *план 2*, так как фильтрация данных выполняется до соединения таблиц, что уменьшает количество кортежей для последующей обработки
- Использование *Hash Join* вместо вложенных циклов не сильно сократит время выполнения запроса, а только создаст дополнительные расходы на хэш-таблицу, так как объём данных после фильтрации на самом деле совсем небольшой (0 кортежей)

При добавлении индексов планы выполнения запросов изменятся:
- Вместо полного сканирования таблиц будет использоваться скан по индексу
- Операция LEFT OUTER JOIN станет быстрее при добавлении индекса на атрибут, по которому присоединяют таблицу «Н_СЕССИЯ», так как мы избавимся от вложенного цикла, но в нашем случае это не критично из-за нулевого количества строк в итоговой выборке

Запрос 2:

План 1:
- Полный скан таблицы «Н_ВЕДОМОСТИ» с фильтрацией по условию «Н_ВЕДОМОСТИ».ЧЛВК_ИД = 105590
- Полный скан таблицы «Н_ЛЮДИ» с фильтрацией по условию «Н_ЛЮДИ».ФАМИЛИЯ > ‘Иванов’
- Соединение таблиц «Н_ВЕДОМОСТИ» и «Н_ЛЮДИ» при помощи *Nested Loops Join* по атрибутам «ЧЛВК_ИД» и «ИД» соответственно

- Соединение таблицы «Н_СЕССИЯ» и результатов предыдущего объединения таблиц при помощи *Nested Loops Join* по атрибутам «ЧЛВК_ИД» и «ИД» соответственно

План 2:
- Полный скан таблицы «Н_ВЕДОМОСТИ» с фильтрацией по условию «Н_ВЕДОМОСТИ».ЧЛВК_ИД = 105590
- Полный скан таблицы «Н_ЛЮДИ» с фильтрацией по условию «Н_ЛЮДИ».ФАМИЛИЯ > ‘Иванов’
- Соединение таблиц «Н_ВЕДОМОСТИ» и «Н_ЛЮДИ» при помощи *Hash Join* (создание хэш-таблички) по атрибутам «ЧЛВК_ИД» и «ИД» соответственно
- Соединение таблицы «Н_СЕССИЯ» и результатов предыдущего объединения таблиц при помощи *Hash Join* по атрибутам «ЧЛВК_ИД» и «ИД» соответственно

План 3:
- Полный скан таблицы «Н_ВЕДОМОСТИ» с фильтрацией по условию «Н_ВЕДОМОСТИ».ЧЛВК_ИД = 105590
- Полный скан таблицы «Н_ЛЮДИ» с фильтрацией по условию «Н_ЛЮДИ».ФАМИЛИЯ > ‘Иванов’
- Соединение таблиц «Н_ВЕДОМОСТИ» и «Н_ЛЮДИ» при помощи *Hash Join* (создание хэш-таблички) по атрибутам «ЧЛВК_ИД» и «ИД» соответственно
- Соединение таблицы «Н_СЕССИЯ» и результатов предыдущего объединения таблиц при помощи *Nested Loops Join* по атрибутам «ЧЛВК_ИД» и «ИД» соответственно

Оптимальный план:
- На самом деле оптимальным планом будет *план 3*, несмотря на использование *Nested Loops Join* во втором соединении, потому что в последней выборке у нас будет небольшой объём данных, а именно 0 кортежей, из-за чего больше пользы будет от вложенных циклов, чем от затрат на создание дополнительной хэш-таблицы для реализации соединения

При добавлении индексов планы выполнения запросов изменятся:
- Вместо полного сканирования таблиц при использовании операций WHERE будет выполнен индексный скан
- Вторая операция RIGHT OUTER JOIN станет быстрее при добавлении индекса на атрибут «ИД» таблицы «Н_ЛЮДИ», так как мы избавимся от вложенного цикла, но в случае первого соединения, если используется план 3, то индекс будет проигнорирован в пользу хэш-таблички, которая используется в *Hash Join*

Вывод команды EXPLAIN ANALYZE

Запрос 1:

QUERY PLAN

```
Nested Loop  (cost=0.28..135.93 rows=1 width=17) (actual time=0.508..0.509
rows=0 loops=1)
  ->  Seq Scan on "H_СЕССИЯ" ses  (cost=0.00..127.28 rows=1 width=8) (actual
time=0.507..0.508 rows=0 loops=1)
        Filter: (("ИД" < 1975) AND ("ИД" = 32199))
        Rows Removed by Filter: 3752
  ->  Index Scan using "ЧЛВК_PK" on "H_ЛЮДИ" people  (cost=0.28..8.30 rows=1
width=17) (never executed)
        Index Cond: ("ИД" = ses."ЧЛВК_ИД")
        Filter: (("ФАМИЛИЯ")::text = 'Иванов'::text)
Planning Time: 1.213 ms
Execution Time: 0.592 ms
```

Запрос 2:

QUERY PLAN

```
Nested Loop  (cost=5.01..273.11 rows=1300 width=28) (actual time=0.125..0.126
rows=0 loops=1)
  ->  Index Scan using "ВЕД_ЧЛВК_FK_IFK" on "H_ВЕДОМОСТИ" veds
(cost=0.29..198.81 rows=65 width=12) (actual time=0.025..0.077 rows=27 loops=1)
        Index Cond: ("ЧЛВК_ИД" = 105590)
  ->  Materialize  (cost=4.72..58.10 rows=20 width=24) (actual time=0.002..0.002
rows=0 loops=27)
        ->  Nested Loop  (cost=4.72..58.00 rows=20 width=24) (actual
time=0.035..0.035 rows=0 loops=1)
              ->  Index Scan using "ЧЛВК_PK" on "H_ЛЮДИ" people  (cost=0.28..8.30
rows=1 width=20) (actual time=0.034..0.034 rows=0 loops=1)
                    Index Cond: ("ИД" = 105590)
                    Filter: (("ФАМИЛИЯ")::text > 'Иванов'::text)
                    Rows Removed by Filter: 1
              ->  Bitmap Heap Scan on "H_СЕССИЯ" ses  (cost=4.44..49.50 rows=20
width=8) (never executed)
                    Recheck Cond: ("ЧЛВК_ИД" = 105590)
                    ->  Bitmap Index Scan on "SYS_C003500_IFK"  (cost=0.00..4.43
rows=20 width=0) (never executed)
                          Index Cond: ("ЧЛВК_ИД" = 105590)
Planning Time: 1.357 ms
Execution Time: 0.218 ms
```

Выводы по работе

В ходе выполнения лабораторной работы №4 я познакомился с понятием индексов, прочитал про их различные реализации и узнал, как использовать их для оптимизации запросов к базе данных. Научился строить планы выполнения запросов, узнал как устроен их выбор в СУБД. Познакомился с командой EXPLAIN ANALYZE и усвоил, как анализировать её вывод.